SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. G. A. Senthil

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Descriptive Experiment Exam **Duration: 2hrs**

Code of Conduct:

I ***ALLAPAREDDY JAHNAVI (192472214)*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Jahnavi

EXPERIMENT 1

Familiarization with OpenAI Gym/Gymnasium Environments

Aim

To create, initialize, and interact with the FrozenLake environment using Gymnasium and understand states, actions, rewards, and episodes.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

print("Initial State:", state)

for step in range(10):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"\nStep {step+1}")
    print("Action Taken:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    state = next_state

    if terminated or truncated:
        print("Episode Finished")
        break

env.close()
```

Sample Output

Initial State: 0

Step 1

Action Taken: 2

Next State: 0

Reward: 0

Step 2

Action Taken: 0

Next State: 0

Reward: 0

Step 3

Action Taken: 3

Next State: 1

Reward: 0

Step 4

Action Taken: 0

Next State: 1

Reward: 0

Step 5

Action Taken: 3

Next State: 0

Reward: 0

Step 6

Action Taken: 0

Next State: 4

Reward: 0

Step 7

Action Taken: 0

Next State: 4

Reward: 0

Step 8

Action Taken: 0

Next State: 8

Reward: 0

Step 9

Action Taken: 3

Next State: 8

Reward: 0

Step 10

Action Taken: 0

Next State: 12

Reward: 0

Episode Finished

Observation

- The agent interacted with the FrozenLake environment.
- Different actions moved the agent to different states.
- Rewards were received based on the environment response.

Result

The Gymnasium environment was successfully created and explored.

EXPERIMENT 2

Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework involving agent, environment, state, action, and reward.

Program

```
import gymnasium as gym


env = gym.make("FrozenLake-v1")


state, info = env.reset()
```

```
total_reward = 0

for step in range(20):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    total_reward += reward

    state = next_state

    if terminated or truncated:
        break

print("\nTotal Reward:", total_reward)

env.close()
```

Sample Output

Current State: 0

Action: 0

Next State: 4

Reward: 0

Current State: 4

Action: 3

Next State: 4

Reward: 0

Current State: 4

Action: 0

Next State: 8

Reward: 0

Current State: 8

Action: 0

Next State: 8

Reward: 0

Current State: 8

Action: 2

Next State: 12

Reward: 0

Total Reward: 0

Observation

- The agent continuously interacted with the environment.
- The environment returned rewards and new states.
- The RL interaction cycle was observed.

Result

The Reinforcement Learning framework was successfully implemented.

EXPERIMENT 3

Simulation of a Markov Decision Process (MDP)

Aim

To simulate state transitions and rewards in a simple Markov Decision Process.

Program

```
states = ["A", "B", "C"]


current_state = "A"
```

```
for step in range(5):

    print("\nCurrent State:", current_state)

    if current_state == "A":
        current_state = "B"
        reward = 5

    elif current_state == "B":
        current_state = "C"
        reward = 10

    else:
        current_state = "A"
        reward = 2

    print("Reward:", reward)
    print("Next State:", current_state)
```

Sample Output

Current State: A

Reward: 5

Next State: B

Current State: B

Reward: 10

Next State: C

Current State: C

Reward: 2

Next State: A

Current State: A

Reward: 5

Next State: B

Current State: B

Reward: 10

Next State: C

Observation

- State transitions occurred sequentially.
- Rewards were assigned based on transitions.
- MDP behavior was successfully simulated.

Result

The Markov Decision Process was successfully simulated.

EXPERIMENT 4

Bellman Equation Demonstration

Aim

To demonstrate the Bellman Expectation Equation for value function estimation.

Program

```
gamma = 0.9

rewards = [5, 10]

probabilities = [0.5, 0.5]

future_values = [20, 40]

value = 0

for p, r, fv in zip(probabilities, rewards, future_values):

    value += p * (r + gamma * fv)

print("State Value =", value)
```

Output

State Value = 34.5

Observation

- Future rewards were discounted using γ.
- The Bellman Equation estimated the value of a state.

Result

The Bellman Expectation Equation was successfully demonstrated.

EXPERIMENT 5

Reward Visualization in Reinforcement Learning

Aim

To visualize cumulative rewards obtained by an RL agent using Matplotlib.

Program

```
import matplotlib.pyplot as plt
import random

episodes = list(range(1, 21))

rewards = []

total_reward = 0

for episode in episodes:

    total_reward += random.randint(0, 1)

    rewards.append(total_reward)

plt.plot(episodes, rewards, marker='o')

plt.title("Cumulative Reward vs Episodes")
```

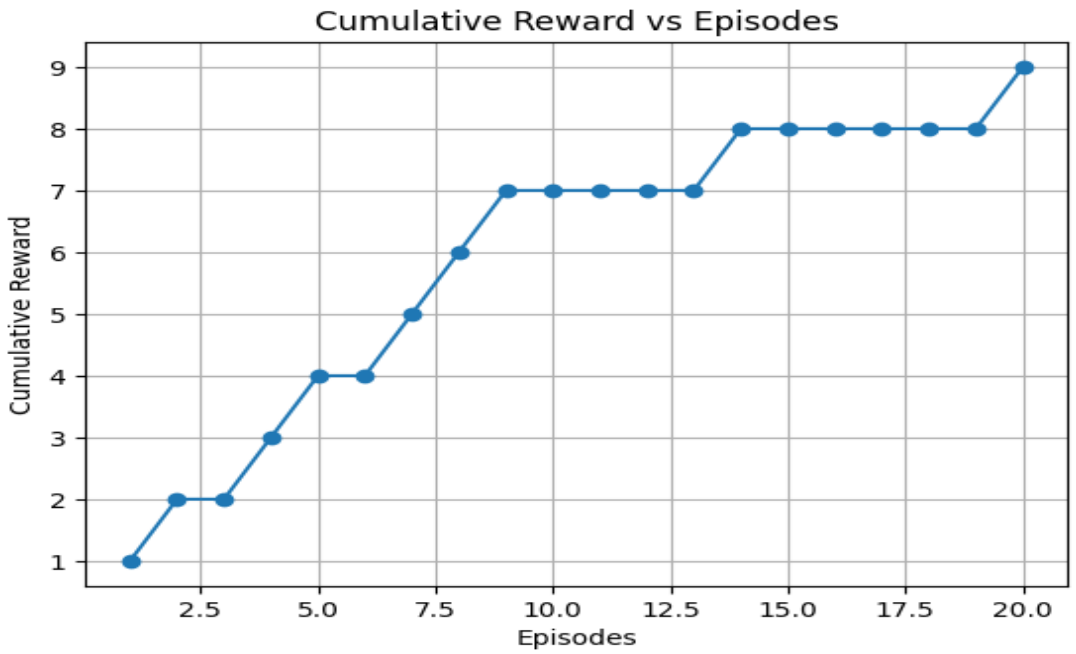
```
plt.xlabel("Episodes")

plt.ylabel("Cumulative Reward")

plt.grid(True)

plt.show()
```

Output



Observation

- The graph displayed learning progress over episodes.
- Cumulative rewards increased gradually.

Result

Reward visualization was successfully performed using Matplotlib.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand